

NAME : RIDDHI PATEL

ENROLLMENT: 12402080601115

CLASS : IT-B

SUBJECT : JAVA(2020044502)

BATCH : 2B8

TOPIC : ASSIGNMENT-1

GITHUB LINK :

https://github.com/riddhi280806/java_program_assignment.git

Assignment-1

1.Implement Array and String operations (Reverse, Sort, Search, Average, Maximum) using class and objects

```
import java.util.*;
class Program1 {
    int[] arr;
    int n;
    // Constructor
    Program1(int size) {
        n = size;
        arr = new int[n];
    }
    // Input method
    void input() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter " + n + " elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
    }
    // Reverse array
    void reverse() {
        System.out.print("Reversed Array: ");
        for (int i = n - 1; i >= 0; i--) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
    // Sort array (Ascending)
    void sort() {
        Arrays.sort(arr);
        System.out.print("Sorted Array: ");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
}
```

```
void search(int key) {
    boolean found = false;
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            System.out.println("Element found at position " + (i + 1));
            found = true;
            break;
        }
    }
    if (!found) {
        System.out.println("Element not found");
    }
}

void average() {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    double avg = (double) sum / n;
    System.out.println("Average = " + avg);
}

void maximum() {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    System.out.println("Maximum = " + max);
}

void reverseString(String str) {
    String rev = "";
    for (int i = str.length() - 1; i >= 0; i--) {
        rev += str.charAt(i);
    }
    System.out.println("Reversed String: " + rev);
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter size of array: ");
    int size = sc.nextInt();
    Program1 obj = new Program1(size);

    obj.input();
    obj.reverse();
    obj.sort();

    System.out.print("Enter element to search: ");
    int key = sc.nextInt();
    obj.search(key);

    obj.average();
    obj.maximum();

    sc.nextLine(); // clear buffer
    System.out.print("Enter a string: ");
    String str = sc.nextLine();

    obj.reverseString(str);
}
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>java Program1
Enter size of array: 5
Enter 5 elements:
11
12
13
14
15
Reversed Array: 15 14 13 12 11
Sorted Array: 11 12 13 14 15
Enter element to search: 13
Element found at position 3
Average = 13.0
Maximum = 15
Enter a string: hello
Reversed String: olleh
```

2. Develop Matrix class with constructors, transpose and multiplication

```
import java.util.*;
class Matrix {
    int rows, cols;
    int[][] mat;
    Matrix(int r, int c) {
        rows = r;
        cols = c;
        mat = new int[rows][cols];
    }
    void input() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter elements:");
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                mat[i][j] = sc.nextInt();
            }
        }
    }
    void display() {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print(mat[i][j] + " ");
            }
            System.out.println();
        }
    }
    Matrix transpose() {
        Matrix t = new Matrix(cols, rows);
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                t.mat[j][i] = mat[i][j];
            }
        }
        return t;
    }
    Matrix multiply(Matrix m) {
        if (this.cols != m.rows) {
```

```
System.out.println("Multiplication not possible");
    return null;
}
Matrix result = new Matrix(this.rows, m.cols);
for (int i = 0; i < this.rows; i++) {
    for (int j = 0; j < m.cols; j++) {
        result.mat[i][j] = 0;
        for (int k = 0; k < this.cols; k++) {
            result.mat[i][j] += this.mat[i][k] * m.mat[k][j];
        }
    }
}
return result;
}
}

public class Program2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter rows and columns of Matrix 1: ");
        int r1 = sc.nextInt();
        int c1 = sc.nextInt();
        Matrix m1 = new Matrix(r1, c1);
        m1.input();
        System.out.print("Enter rows and columns of Matrix 2: ");
        int r2 = sc.nextInt();
        int c2 = sc.nextInt();
        Matrix m2 = new Matrix(r2, c2);
        m2.input();
        System.out.println("\nMatrix 1:");
        m1.display();
        System.out.println("\nMatrix 2:");
        m2.display();
        System.out.println("\nTranspose of Matrix 1:");
        Matrix t = m1.transpose();
        t.display();

        // Multiplication
        System.out.println("\nMultiplication Result:");
```

```
Matrix result = m1.multiply(m2);
    if (result != null) {
        result.display();
    }
}
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>java Program2
Enter rows and columns of Matrix 1: 2 2
Enter elements:
12
13
14
15
Enter rows and columns of Matrix 2: 2 2
Enter elements:
21
22
23
24

Matrix 1:
12 13
14 15

Matrix 2:
21 22
23 24

Transpose of Matrix 1:
12 14
13 15

Multiplication Result:
551 576
639 668
```

3. Write a program demonstrating Wrapper classes and String vs StringBuffer

```
class Program3 {
    public static void main(String[] args) {

        // ===== Wrapper Classes =====
        int num = 10;

        // Boxing (primitive to object)
        Integer obj = Integer.valueOf(num);
        System.out.println("Boxing: " + obj);

        // Unboxing (object to primitive)
        int num2 = obj.intValue();
        System.out.println("Unboxing: " + num2);

        // Autoboxing & Auto-unboxing
        Integer autoObj = num; // autoboxing
        int autoNum = autoObj; // auto-unboxing
        System.out.println("Autoboxing: " + autoObj);
        System.out.println("Auto-unboxing: " + autoNum);

        // ===== String vs StringBuffer =====

        // String (Immutable)
        String s1 = "Hello";
        s1.concat(" World"); // does not change original string
        System.out.println("\nString after concat: " + s1);

        // StringBuffer (Mutable)
        StringBuffer sb = new StringBuffer("Hello");
        sb.append(" World"); // changes original object
        System.out.println("StringBuffer after append: " + sb);

        // Performance demonstration
        long start, end;

        // String performance
```



```
start = System.currentTimeMillis();
String s = "";
for (int i = 0; i < 10000; i++) {
    s += "a";
}
end = System.currentTimeMillis();
System.out.println("\nTime taken by String: " + (end - start) + " ms");

// StringBuffer performance
start = System.currentTimeMillis();
StringBuffer sb2 = new StringBuffer("");
for (int i = 0; i < 10000; i++) {
    sb2.append("a");
}
end = System.currentTimeMillis();
System.out.println("Time taken by StringBuffer: " + (end - start) + " ms");
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>javac Program3.java
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>java Program3
Boxing: 10
Unboxing: 10
Autoboxing: 10
Auto-unboxing: 10

String after concat: Hello
StringBuffer after append: Hello World

Time taken by String: 51 ms
Time taken by StringBuffer: 2 ms
```

4.Implement BankAccount class with deposit, withdraw and balance inquiry

```
import java.util.*;

class BankAccount {
    private double balance;

    // Constructor
    BankAccount(double initialBalance) {
        balance = initialBalance;
    }

    // Deposit method
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: " + amount);
        } else {
            System.out.println("Invalid deposit amount");
        }
    }

    // Withdraw method
    void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient balance or invalid amount");
        }
    }

    // Balance inquiry
    void checkBalance() {
        System.out.println("Current Balance: " + balance);
    }
}

public class Program4 {
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter initial balance: ");
    double initial = sc.nextDouble();

    BankAccount acc = new BankAccount(initial);

    int choice;

    do {
        System.out.println("\n1. Deposit");
        System.out.println("2. Withdraw");
        System.out.println("3. Check Balance");
        System.out.println("4. Exit");
        System.out.print("Enter choice: ");

        choice = sc.nextInt();

        switch (choice) {
            case 1:
                System.out.print("Enter amount to deposit: ");
                double d = sc.nextDouble();
                acc.deposit(d);
                break;

            case 2:
                System.out.print("Enter amount to withdraw: ");
                double w = sc.nextDouble();
                acc.withdraw(w);
                break;

            case 3:
                acc.checkBalance();
                break;

            case 4:
                System.out.println("Thank you!");
                break;
        }
    }
}
```

```
default:
    System.out.println("Invalid choice");
}

} while (choice != 4);
}
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>javac Program4.java

C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>java Program4
Enter initial balance: 10000

1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter choice: 3
Current Balance: 10000.0

1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter choice: 4
Thank you!
```

5.Develop inheritance-based Cricket–Match system using command line arguments

```
import java.util.Scanner;
```

```
class Cricket {  
    String team1, team2;  
  
    Cricket(String t1, String t2) {  
        team1 = t1;  
        team2 = t2;  
    }  
  
    void showTeams() {  
        System.out.println("Match Between: " + team1 + " vs " + team2);  
    }  
}
```

```
class Match extends Cricket {  
    int score1, score2;  
  
    Match(String t1, String t2, int s1, int s2) {  
        super(t1, t2);  
        score1 = s1;  
        score2 = s2;  
    }  
  
    void result() {  
        if (score1 > score2)  
            System.out.println(team1 + " wins!");  
        else if (score2 > score1)  
            System.out.println(team2 + " wins!");  
        else  
            System.out.println("Match Draw!");  
    }  
  
    void display() {  
        showTeams();  
        System.out.println(team1 + " Score: " + score1);  
    }  
}
```

```
        System.out.println(team2 + " Score: " + score2);
        result();
    }
}
```

```
class Program5 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Team 1: ");
        String t1 = sc.nextLine();

        System.out.print("Enter Team 2: ");
        String t2 = sc.nextLine();

        System.out.print("Enter Score of " + t1 + ": ");
        int s1 = sc.nextInt();

        System.out.print("Enter Score of " + t2 + ": ");
        int s2 = sc.nextInt();

        Match m = new Match(t1, t2, s1, s2);
        m.display();
    }
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>java Program5
Enter Team 1: India
Enter Team 2: England
Enter Score of India: 255
Enter Score of England: 242
Match Between: India vs England
India Score: 255
England Score: 242
India wins!
```

6.Implement Cipher system using Abstract class and method overriding

```
import java.util.Scanner;
abstract class Cipher {
    String text;
    Cipher(String text) {
        this.text = text;
    }
    abstract String encrypt();
    abstract String decrypt();
}
class CaesarCipher extends Cipher {
    int shift;
    CaesarCipher(String text, int shift) {
        super(text);
        this.shift = shift;
    }

    String encrypt() {
        String result = "";
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (Character.isLetter(ch)) {
                char base = Character.isUpperCase(ch) ? 'A' : 'a';
                ch = (char) ((ch - base + shift) % 26 + base);
            }
            result += ch;
        }
        return result;
    }

    String decrypt() {
        String result = "";
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (Character.isLetter(ch)) {
                char base = Character.isUpperCase(ch) ? 'A' : 'a';
                ch = (char) ((ch - base - shift + 26) % 26 + base);
            }
            result += ch;
        }
    }
}
```

```
result += ch;
}
return result;
}
}

class Program6 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter text: ");
        String text = sc.nextLine();
        System.out.print("Enter shift value: ");
        int shift = sc.nextInt();
        CaesarCipher c = new CaesarCipher(text, shift);
        String enc = c.encrypt();
        System.out.println("Encrypted: " + enc);

        CaesarCipher d = new CaesarCipher(enc, shift);
        System.out.println("Decrypted: " + d.decrypt());
    }
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>javac Program6.java

C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>java Program6
Enter text: i love India
Enter shift value: 4
Encrypted: m pszi Mrhme
Decrypted: i love India
```


7.Demonstrate Inner Classes (member, local, anonymous)

```
class Outer {  
    // Member Inner Class  
    class MemberInner {  
        void show() {  
            System.out.println("This is Member Inner Class");  
        }  
    }  
    void display() {  
        // Local Inner Class  
        class LocalInner {  
            void show() {  
                System.out.println("This is Local Inner Class");  
            }  
        }  
        LocalInner l = new LocalInner();  
        l.show();  
    }  
}  
interface Demo {  
    void show();  
}  
public class Program7 {  
    public static void main(String[] args) {  
        Outer obj = new Outer();  
        Outer.MemberInner m = obj.new MemberInner();  
        m.show();  
        obj.display();  
        Demo d = new Demo() {  
            public void show() {  
                System.out.println("This is Anonymous Inner Class");  
            }  
        };  
        d.show();  
    }  
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>javac Program7.java  
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>java Program7  
This is Member Inner Class  
This is Local Inner Class  
This is Anonymous Inner Class
```

8.Create custom exception handling for bank withdrawal scenario

```
// Custom Exception
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

// Bank Account class
class BankAccount {
    double balance;

    BankAccount(double balance) {
        this.balance = balance;
    }

    // Withdraw method with exception
    void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException("Insufficient Balance!");
        } else {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
            System.out.println("Remaining Balance: " + balance);
        }
    }
}

// Main class
public class Program8 {
    public static void main(String[] args) {

        BankAccount acc = new BankAccount(1000);

        try {
            acc.withdraw(1500); // trying to withdraw more than balance
        } catch (InsufficientBalanceException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

```
try {  
    acc.withdraw(500); // valid withdrawal  
} catch (InsufficientBalanceException e) {  
    System.out.println("Exception: " + e.getMessage());  
}  
}  
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>javac Program8.java  
  
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-1>java Program8  
Exception: Insufficient Balance!  
Withdrawn: 500.0  
Remaining Balance: 500.0
```